

Practical Week 11

1. Develop a multithreaded counter application using POSIX threads (pthread_create, pthread_join). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.

```
imvarls@LAPTOP-VQ4D70L7: × + ▾
GNU nano 8.7.1 race_condition.c *
#include <stdio.h>
#include <pthread.h>

#define NUM_THREADS 4
#define ITERATIONS 100000

int counter = 0;

void *increment(void *arg) {
    for (int i = 0; i < ITERATIONS; i++) {
        counter++;
    }
    return NULL;
}

int main() {
    pthread_t threads[NUM_THREADS];

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_create(&threads[i], NULL, increment, NULL);
    }

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("Expected counter: %d\n", NUM_THREADS * ITERATIONS);
    printf("Actual counter: %d\n", counter);

    return 0;
}
```

Output:

```
imvarls@LAPTOP-VQ4D70L7: × + ▾
imvarls@LAPTOP-VQ4D70L7:~$ nano race_condition.c
imvarls@LAPTOP-VQ4D70L7:~$ gcc race_condition.c -o race_condition
imvarls@LAPTOP-VQ4D70L7:~$ ./race_condition
Expected counter: 400000
Actual counter: 276891
imvarls@LAPTOP-VQ4D70L7:~$ |
```

2. Modify the previous program using mutex locks (`pthread_mutex_lock`, `pthread_mutex_unlock`) and compare the outputs with the race-condition version. Analyze the impact of synchronization.

```
imvarls@LAPTOP-VQ4D70L7: × + v
GNU nano 8.7.1 mutex_counter.c *

#include <stdio.h>
#include <pthread.h>

#define NUM_THREADS 4
#define ITERATIONS 100000

int counter = 0;
pthread_mutex_t lock;

void *increment(void *arg) {
    for (int i = 0; i < ITERATIONS; i++) {
        pthread_mutex_lock(&lock);
        counter++;
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

int main() {
    pthread_t threads[NUM_THREADS];

    pthread_mutex_init(&lock, NULL);

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_create(&threads[i], NULL, increment, NULL);
    }

    for (int i = 0; i < NUM_THREADS; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("Expected counter: %d\n", NUM_THREADS * ITERATIONS);
    printf("Actual counter: %d\n", counter);

    pthread_mutex_destroy(&lock);

    return 0;
}
```

Output:

```
imvarls@LAPTOP-VQ4D70L7:~$ gcc mutex_counter.c -o mutex_counter -pthread
imvarls@LAPTOP-VQ4D70L7:~$ ./mutex_counter
Expected counter: 400000
Actual counter: 400000
imvarls@LAPTOP-VQ4D70L7:~$ |
```